

Prioritized Study Plan for C++ Features

Introduction

This document lists key C++ features to study, with a focus on threading and safety enhancements, organized by C++ version. Each feature is accompanied by a brief description to guide your study. The topics are categorized into high, medium, and low priority based on their relevance to modern C++ development and your goals.

High Priority Topics

Threading and Concurrency

- **`std::thread`**: Introduces threading support in the standard library, enabling concurrent execution of code.
- **`std::mutex`, `std::lock`, `std::unique_lock`**: Provides mechanisms for managing access to shared resources and preventing race conditions.
- **`std::atomic`**: Introduces atomic operations to ensure thread safety without using mutexes, providing a performance boost in some cases.
- **`std::jthread`**: A more convenient thread class that automatically joins upon destruction, reducing the risk of resource leaks.
- **`std::scoped_lock`**: Provides a convenient way to lock multiple mutexes simultaneously, preventing deadlocks.
- **Efficient Concurrency**: Techniques such as lock-free programming and advanced locking mechanisms to improve concurrency performance.
- **Memory Models and Concurrency**: Understanding how memory models interact with concurrent programming, including memory access in multi-threaded programs.

Coroutines and Asynchronous Programming

- **Coroutines**: Enables writing asynchronous code in a synchronous style, simplifying the management of asynchronous tasks.

- **Structured Concurrency with Senders and Receivers:** A modern approach to managing asynchronous processing pipelines and data parallelism.
- **Async with Coroutines:** Using coroutines for asynchronous tasks, focusing on `async`, `await`, and structured concurrency.

Concepts and Templates

- **Concepts:** Introduces constraints for templates, improving the readability and safety of template code.
- **Generic Functions with `auto`:** Simplifies the definition of generic functions by using `auto` as a type placeholder.

Ranges and Views

- **Ranges:** Provides a new way to work with sequences of data, offering a safer and more expressive alternative to traditional iterators.
- **Views in Detail:** Detailed study of view types like `transform_view`, `filter_view`, and `iota_view`, which are essential for modern C++ programming.

Medium Priority Topics

Memory and Performance

- **`std::unique_ptr`, `std::shared_ptr`:** Introduces smart pointers to automate memory management and reduce the risk of memory leaks and dangling pointers.
- **move semantics and rvalue references:** Enhances performance by enabling efficient transfer of resources, minimizing unnecessary copying.
- **Formatted Output (`std::format`):** Using `format()`, `format_to()`, and user-defined formatters for modern string formatting.

Modules and Compile-Time Improvements

- **Modules:** Facilitates better modularization of code, reducing compilation times and improving code organization.
- **Compile-Time Enhancements:** Topics like `constexpr`, `constexpr`, and compile-time computations with complex data structures.

Low Priority Topics

Other Language Features

- **Spaceship Operator `<=>`:** Introduces three-way comparison, allowing easier implementation of comparison operators.
- **Chrono Extensions:** Calendar dates and time zones, providing advanced time and date handling.
- **String and Bits Extensions:** Extensions for handling strings and bit-wise operations, including `std::span` and bit manipulation utilities.

Additional Resources

- **cppreference.com:** Comprehensive reference for C++ features, including details on the latest standards.
- **Clang Website:** Lists C++ features supported by the Clang compiler, useful for understanding what is available and implemented.